

Tell me about a time you faced a technical challenge and how you overcame it

S: While building a banking API for a personal project, I discovered the endpoints were completely open - anyone could access account services like deposits and transfers without authentication.

T: I needed to secure the API so that only authenticated users could access their accounts - critical for a banking application.

A: I implemented JWT authentication. When a user logs in, they're issued a token, and I built a custom filter that validates that token on every request before allowing access. The trickiest part was getting the filter to fit correctly in Spring Security's filter chain, so I worked through how requests are processed to place it correctly.

R: After that, only authenticated users could access their accounts, and every request was verified. It taught me a lot about how security actually works in a real application.

Describe a time you worked on a team. What was your role?

S: On my de-identification project, I worked on a team building an app to de-identify sensitive data.

T: I was responsible for the frontend and keeping our version control organized.

A: I built the React frontend and managed our Git workflow, and I coordinated closely with my teammate on the Python and AWS backend to make sure our data formats lined up.

R: We got a working app delivered, and it taught me you can't work in isolation - you have to communicate constantly with your team.

Tell me about a disagreement or conflict with a teammate.

S: On that same project, my teammate and I ran into a problem where the data format the backend was sending didn't match what my frontend expected.

T: We needed to agree on a shared format to get the app working.

A: Instead of both of us pushing our own way, I sat down with him and we walked through both sides, understood each other's constraints, and agreed on a common format.

R: We fixed it pretty quickly, and it showed me that most conflicts are really just coordination problems you solve by talking them through.

Describe a time you had to learn something new quickly.

S: When I was building my e-commerce app, I needed to use Kafka for messaging between services, which I hadn't worked with before.

T: I had to understand it well enough to actually use it in my project.

A: I broke it down, read the documentation, tested small pieces, and kept iterating until it clicked.

R: I got it working for the notification service, and it reminded me I can pick up new technologies pretty quickly by breaking them into smaller parts.

Tell me about a mistake you made and how you handled it.

S: On my banking project, I realized two withdrawals could happen at the same time and cause a race condition on the account balance - something I hadn't accounted for at first.

T: I needed to fix it so transactions were handled safely, without breaking anything else.

A: I traced the issue, reproduced it, and fixed it using locking so the transactions would be processed safely one at a time.

R: It worked, and it reinforced my habit of writing code and then re-reading and testing it carefully to catch things I miss.

Describe a time you had to meet a tight deadline.

S: When I was wrapping up one of my projects, I had a limited amount of time to get it fully working.

T: I needed to deliver something that worked without running out of time.

A: I prioritized the core features first, broke the work into smaller pieces, and focused on getting a working version before going back to refine it.

R: I got it done on time with the main functionality working, and it taught me to focus on the essentials first when I'm under time pressure.

Tell me about a time you solved a difficult problem.

S: In my e-commerce project, I needed the services to talk to each other, but some operations couldn't make the user wait while others needed an immediate answer.

T: I had to design the communication so it was both responsive and reliable.

A: I used synchronous REST for the order-to-inventory check, since the order needs to confirm stock before going through, and asynchronous Kafka messaging for notifications, since those shouldn't block the order and should still go through even if the service is temporarily down.

R: The system ended up both fast and resilient, and it taught me to match the communication style to what each operation actually needs.

Describe a project you're proud of.

S: I built an e-commerce application using microservices architecture.

T: I wanted to build something that reflected real-world, scalable design, not just a basic app.

A: I designed three services - order, inventory, and notification - communicating through REST and Kafka, with JWT authentication and an Angular frontend, and I made deliberate choices about when to use synchronous versus asynchronous communication.

R: I ended up with a working, scalable app that shows real architectural thinking, and it's the project I'm most proud of.

Tell me about a time you received difficult feedback.

S: During a code review on one of my projects, I got feedback that some of my code was harder to follow than it needed to be.

T: I needed to take it constructively and actually improve the code.

A: I listened without getting defensive, asked a couple of questions to make sure I understood the feedback, and went back and cleaned up my code.

R: The code came out clearer and better, and it helped me start seeing feedback as something that makes my work better rather than criticism.

Why do you want to work in this role?

S: As a recent CS grad, I'm looking to launch my career and get real, hands-on experience.

T: I want a role where I can apply my foundation in Java and full-stack development while continuing to learn.

A: I bring solid fundamentals, real project experience, and a genuine eagerness to grow and contribute on a real team.

R: My goal is to keep developing into a strong engineer and contribute to real-world solutions that make an impact.